

**AUTOCODE ROVER (AN AUTONOMOUS PROGRAM
IMPROVEMENT SYSTEM)**

A Project Report submitted in partial fulfillment of the requirements for the award of the
degree of

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted by

N RAGHAVENDRA (HU22CSEN0101032)

P THIRUMALA SURESH (HU22CSEN0101235)

B MOUNIKA REDDY (HU22CSEN0101432)

P SAMPATH REDDY (HU22CSEN0101482)

Under the esteemed guidance of

JINKA SREEDHAR

ASSOCIATE PROFESSOR



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GITAM SCHOOL OF TECHNOLOGY
GITAM (Deemed to be University)
HYDERABAD**

2025

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GITAM SCHOOL OF TECHNOLOGY
GITAM (Deemed to be University)**



DECLARATION

I hereby declare that the project report entitled **AUTOCODE ROVER (AN AUTONOMOUS PROGRAM IMPROVEMENT SYSTEM)** is an original work done in the Department of Computer Science and Engineering, GITAM School of Technology, GITAM (Deemed to be University) submitted in partial fulfillment of the requirements for the award of the degree of B.Tech. in Computer Science Engineering. The work has not been submitted to any other college or University for the award of any degree or diploma.

Date:

Registration No(s)	Name(s)	Signature
HU22CSEN0101235	P THIRUMALA SURESH	
HU22CSEN0101032	N RAGHAVENDRA	
HU22CSEN0101432	B MOUNIKA REDDY	
HU22CSEN0101482	P SAMPATH REDDY	

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GITAM SCHOOL OF TECHNOLOGY
GITAM (Deemed to be University)**



CERTIFICATE

This is to certify that the project report entitled “**AUTOCODE ROVER (AN AUTONOMOUS PROGRAM IMPROVEMENT SYSTEM)**” is a bonafide record work carried out by **P THIRUMALA SURESH(HU22CSEN0101235), N RAGHAVENDRA(HU22CSEN0101032),B MOUNIKA REDDY (HU22CSEN0101432), P SAMPATH REDDY(HU22CSEN0101482)** students submitted in partial fulfilment of requirement for the award of degree of Bachelors of Technology in Computer Science and Engineering.

Date :

Project Guide

JINKA SREEDHAR

ASSOCIATE PROFESSOR

Head of the Department

SHAIK MAHABOOB BASHA

HOD (CSE)

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to everyone who supported us during the completion of this project, "**AutoCodeRover: Autonomous Program Improvement.**"

We are profoundly thankful to our Project Guide **JINKA SREEDHAR, ASSOCIATE PROFESSOR**, for their expert advice, valuable time, and continuous encouragement. Their guidance was instrumental in the success of this project.

We also thank the **Head of the Department, Computer Science and Engineering**, for providing the necessary facilities and a supportive environment for our work at the **GITAM School of Technology, GITAM (Deemed to be University)**.

TABLE OF CONTENTS

S.No.	Description	Page No.
1.	Abstract	1
2.	Introduction	2
	2.1.... Overview	
	2.2.... Problem Justification	
3.	Literature Review	4
	3.1.... Automated Program Repair(APR)	
	3.2.... LLM Agents for software engineering	
4.	Problem Identification & Objectives	7
	4.1.... Problem Identification	
	4.2.... Objectives	
5.	Existing System	10
	5.1.... SWE-AGENT	
	5.2.... Drawbacks of the existing system	
6.	Proposed System	12
	6.1.... AUTOCODEROVER: A Structure Aware Approach	
	6.2... Two Stage Workflow	
	6.2.1.... Context Retrieval	
	6.2.2.... Patch Generation	
7.	System Architecture	15
	7.1.... Architectural Components	
	7.2.... Use Case Diagram and Description	

7.2.1.... Use Case Diagram

7.2.2.... Description of the Project Flow

8. Tools/Technologies Used 18

8.1.... Large Language Models(LLM)

8.2.... Abstract Syntax Tree(AST) Analysis

8.3.... Spectrum Based Fault Localization(SBFL)

8.4.... Python and Docker

9. Conclusion 21

10. References 22

1. ABSTRACT

AutoCodeRover is a fully automated approach designed for **autonomous program improvement** by resolving real-life GitHub issues for tasks such as bug fixing and feature addition. It addresses the challenge of insufficient context in existing Large Language Model (LLM) agents by adopting a software engineering-oriented approach.

The system's core innovation lies in its use of **Program Structure Aware** code search, which utilizes Abstract Syntax Tree (AST) analysis to identify relevant methods and classes, instead of treating the codebase as a mere collection of files. AutoCodeRover operates in two stages: iterative context retrieval and patch generation, and it can optionally leverage **Spectrum-Based Fault Localization (SBFL)** using test suites to further sharpen the context.

Experiments on the SWE-bench lite dataset demonstrated its effectiveness, achieving 19% efficacy (pass@1). Notably, it resolves issues quickly and economically, with an average time of approximately 4 minutes per task and a cost of around \$0.43 USD, contrasting sharply with the average manual developer time of over 2.68 days. This workflow enables autonomous software engineering by ensuring that modifications are guided by program specifications gleaned from the code structure.

2. INTRODUCTION

2.1 OVERVIEW

Software engineering involves the critical process of **program improvement**, which encompasses software maintenance (e.g., fixing bugs) and software evolution (e.g., adding features). While Large Language Models (LLMs) have significantly advanced automated coding, the goal of fully autonomous program improvement remains a major research focus.

The **AutoCodeRover** project proposes an automated, LLM-based approach to resolve real-life GitHub issues and autonomously achieve this program improvement. It integrates LLMs with sophisticated, structural code search capabilities to generate accurate program modifications, or patches.

The project is particularly distinguished by its **software engineering oriented outlook**, which focuses on program representations rather than a simple collection of files.

1.2 PROBLEM JUSTIFICATION

The development of autonomous LLM agents for software engineering has been hampered by the challenge of providing the agent with the correct and relevant context from a large codebase. Simply searching for code snippets using string matching is ineffective in complex projects.

The need is for a system that can understand the **program structure** (classes, methods) to effectively localize faults and guide the LLM's patching strategy. AutoCodeRover is justified by its novel approach of using **Abstract Syntax Tree (AST)** analysis for code search, which directly addresses this contextual challenge.

3. LITERATURE REVIEW

3.1 AUTOMATED PROGRAM REPAIR (APR)

Test-suite based Automated Program Repair (APR) has been a significant area of research for the last decade, aiming to generate patches that pass a given test suite. These techniques are generally categorized into search-based (e.g., GenProg), semantics-based, and learning-based approaches. A recent and growing line of work involves using LLMs for APR, often assuming the buggy locations are already known (the "perfect fault localization" assumption). However, obtaining the correct buggy locations in a large project remains an essential and challenging task that traditional APR techniques struggle with.

3.2 LLM AGENTS FOR SOFTWARE ENGINEERING

The emergence of the SWE-bench lite benchmark, which consists of 300 real-life software engineering tasks from large Python projects, has spurred the development of LLM agents. These tasks involve complex steps like reasoning about bug locations, analyzing root causes, and writing patches based only on the issue description.

Key LLM-based agents in this space include:

- **Devin:** A closed-source commercial tool that was reported to solve various software engineering tasks.
- **SWE-Agent:** A concurrent work that designed an Agent-Computer Interface (ACI) to allow the LLM to execute basic file operations via shell commands for interaction with the code repository.

AutoCodeRover is distinct because it posits an approach that gleans specification from **software structure** to guide patching, contrasting with the approach of simply viewing the codebase as a collection of files, as is common in other LLM agents.

4. PROBLEM IDENTIFICATION & OBJECTIVES

4.1 PROBLEM IDENTIFICATION

The core issue in achieving autonomous software engineering with LLMs is the failure to effectively bridge the gap between a natural language issue description and the voluminous, structurally complex source code.

The specific identified problems are:

1. **Lack of Program Structure Awareness:** Many LLM agents treat a software project as a set of files and use simple string matching for context retrieval. This leads to retrieving irrelevant context and poor fault localization.
2. **Challenging Fault Localization:** For large, real-life software projects, obtaining the precise buggy location is an essential yet highly challenging task, which many learning-based APR techniques currently side-step by assuming perfect localization.
3. **Inefficiency and Cost:** Existing solutions, such as Devin, have reported results on subsets but often lack transparency on time and token costs, hindering adoption for continuous autonomous improvement.

4.2 OBJECTIVES

The primary objectives defined for the AutoCodeRover project are to:

- **Develop a Program Structure Aware Code Search:** To utilize program representations, specifically the Abstract Syntax Tree (AST), for code search, thereby providing the LLM with relevant classes, methods, and code snippets.
- **Implement an Iterative Context Retrieval Strategy:** To allow the LLM agent to dynamically direct navigation and iteratively refine the code context it retrieves based on previous API call results.
- **Integrate Fault Localization Techniques:** To leverage debugging techniques like Spectrum-Based Fault Localization (SBFL) when a test suite is available, guiding the

search for code and prioritizing context retrieval from methods with high suspicion scores.

- **Achieve State-of-the-Art Efficacy and Efficiency:** To demonstrate high efficacy in solving real-life GitHub issues on the SWE-bench lite benchmark while ensuring the process is time- and cost-efficient.

5. EXISTING SYSTEM

5.1 SWE-AGENT

The SWE-Agent is a notable existing LLM agent designed to tackle tasks in the SWE-bench lite benchmark. It implements an **Agent-Computer Interface (ACI)**, which allows the LLM agent to interact with the software repository using familiar shell commands and file operations.

5.2 DRAWBACKS OF THE EXISTING SYSTEM

The main drawbacks associated with the approach used by SWE-Agent and other similar LLM agents are:

- **View of Codebase as Files:** The agent typically interacts with the codebase as a collection of files and lines of text, fundamentally relying on basic file operations and string matching for navigation and search.
- **Non-Structural Search:** The lack of program structure awareness means the agent may retrieve vast amounts of irrelevant code context. The LLM must then internally process and filter this noise, which can lead to inefficient reasoning and poor localization of the actual bug.
- **Reliance on Shell Commands:** While effective for basic operations, relying on shell commands and file-level operations abstracts away the semantic structure of the code, making the fault localization process unnecessarily complex for the LLM to manage⁴. This approach lacks the intrinsic code-understanding capabilities that a human developer uses when navigating by class and method names.

6. PROPOSED SYSTEM

6.1 AUTOCODEROVER: A STRUCTURE-AWARE APPROACH

The Proposed System, **AutoCodeRover**, is built on the principle of injecting a **software engineering outlook** into the LLM agent workflow. It improves upon existing systems by working directly on a program representation, the **Abstract Syntax Tree (AST)**, to guide the patching process.

6.2 TWO-STAGE WORKFLOW

AutoCodeRover operates in two distinct, sequential stages to resolve a GitHub issue:

6.2.1 Context Retrieval (Localization Stage)

The LLM first analyzes the natural language issue description to extract keywords (files, classes, methods). It then employs a **stratified and iterative strategy** to retrieve code context:

1. **AST-Based Code Search APIs:** The LLM invokes sophisticated code search APIs (e.g., `search_method_in_class()`) that run locally and are powered by AST analysis. These APIs retrieve structured context, such as class signatures and method implementation details.
2. **Iterative Refinement:** The LLM agent directs the navigation, deciding which APIs to use in each iteration based on the context returned from previous calls.
3. **Spectrum-Based Fault Localization (SBFL) Enhancement:** If a test suite is available, AutoCodeRover leverages SBFL, which assigns a suspicion score to different program methods based on test control flow. The LLM agent can prioritize context retrieval from methods identified as highly suspicious by SBFL, thus sharply guiding the search for buggy locations.

6.2.2 Patch Generation (Repair Stage)

Once the retrieval agent has deemed the collected context sufficient, the system proceeds to the Patch Generation stage.

1. **Context Transfer:** A dedicated patch generation agent receives the problem statement, the identified buggy locations/methods, and the full history of context retrieval and analysis.
2. **Patch Drafting and Retry:** The agent first retrieves the precise code snippets at the identified buggy locations. It then enters a retry-loop to generate a patch. If the patch fails syntax checks (e.g., indentation errors) or fails to apply, the agent is prompted to retry up to a pre-configured limit.
3. **Validation:** Finally, the generated patch is validated against the available test suite to ensure correctness.

This system significantly improves over the existing file-based approach by grounding the LLM's reasoning in the semantic structure of the code, leading to faster, more accurate fault localization.

7. SYSTEM ARCHITECTURE

7.1 ARCHITECTURAL COMPONENTS

The AutoCodeRover system is conceptually divided into three main components: the **Issue Analyzer**, the **Context Retrieval Agent**, and the **Patch Generation Agent**, connected to the project's codebase and testing environment.

7.2 USE CASE DIAGRAM AND DESCRIPTION

7.2.1 Use Case Diagram

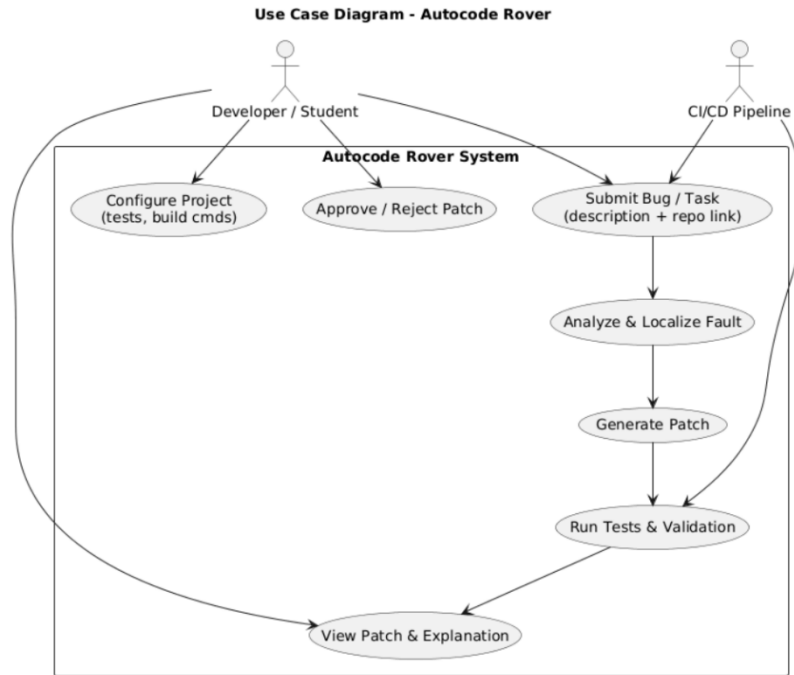


Figure 7.1 : Use Case Diagram of the AutoCodeRover System

7.2.1 Description of the Project Flow

1. **Issue Analysis:** The process begins when the initial LLM agent analyzes the natural language issue description from GitHub to identify potential files, classes, or methods involved.
2. **Context Retrieval:** The **Context Retrieval Agent** then takes the keywords and executes an iterative search. It primarily uses **AST-based Code Search APIs** to retrieve structured context (like method implementations).
3. **Fault Localization (Optional):** If a test suite exists, **Spectrum-Based Fault Localization (SBFL)** is performed, providing suspicion scores to program methods. The agent uses these scores alongside the issue description keywords to prioritize subsequent context retrieval API calls.
4. **Patch Generation:** Once the buggy locations and sufficient context are collected, they are passed to the **Patch Generation Agent**. This agent retrieves the precise code snippets, enters a retry loop, and drafts the final patch.

5. **Validation:** The final patch is validated using the available test cases. If it fails, the patch generation is retried with a limit; otherwise, the correct patch is returned as the solution.

8. TOOLS/TECHNOLOGIES USED

8.1 LARGE LANGUAGE MODELS (LLMs)

The core intelligence of AutoCodeRover is derived from state-of-the-art LLMs.

- **Role:** The LLMs serve as the **agents** for both the Context Retrieval and Patch Generation stages, performing complex reasoning, code analysis, and code generation.
- **Specific Models:** The reported experiments utilized the **OpenAI GPT-4** model (e.g., gpt-4-0125-preview), which is considered state-of-the-art. The project has also been updated to support models from **Gemini, Groq, and Anthropic** (via AWS Bedrock).
- **Significance:** LLMs enable the system to understand ambiguous natural language requirements and generate complex, human-like code modifications autonomously.

8.2 ABSTRACT SYNTAX TREE (AST) ANALYSIS

The most distinguishing technology used is the application of AST for code search.

- **Role:** The AST is a tree representation of the abstract syntactic structure of source code. AutoCodeRover's custom code search APIs run locally based on this AST analysis.
- **Significance:** Using the AST ensures that the context retrieval is **program structure aware**. This is vital for navigating large codebases by precise structural entities like class signatures and method implementation details, rather than relying on inefficient text matching.

8.3 SPECTRUM-BASED FAULT LOCALIZATION (SBFL)

SBFL is a debugging technique leveraged when test cases are provided.

- **Role:** SBFL considers the control flow of passing and failing tests to assign a **suspiciousness score** to different methods in the program.
- **Significance:** This technique provides an execution-based analysis that complements the LLM's reasoning by revealing information not explicitly mentioned in the issue description, thus sharply guiding the agent toward the most likely buggy locations for context retrieval.

8.4 PYTHON AND DOCKER

- **Implementation Language:** The underlying projects being addressed (e.g., Django, SymPy) are primarily Python projects, and the AutoCodeRover codebase and environment management rely on Python.

- **Deployment:** The system is containerized, with an installation setup often recommending **Docker** for running the tool, ensuring a consistent and isolated execution environment.

9. CONCLUSION

AutoCodeRover represents a significant step towards achieving **fully autonomous program improvement** by effectively addressing the challenge of code context retrieval in LLM agents. By consciously adopting a software engineering perspective and integrating **Abstract Syntax Tree (AST) analysis** into its core code search mechanism, the system moves beyond mere file-level operations to accurately target structural entities like classes and methods.

This approach has led to favorable results on the industry-standard SWE-bench lite benchmark, achieving **19% efficacy** while maintaining exceptional time and cost efficiency (solving tasks in approximately 4 minutes at \$0.43 USD). Furthermore, its ability to integrate dynamic debugging techniques like Spectrum-Based Fault Localization (SBFL) demonstrates a robust and versatile architecture for automated program repair.

In conclusion, AutoCodeRover is a robust framework that successfully combines the generative power of LLMs with foundational software engineering principles to enable the autonomous evolution and maintenance of complex software projects.

10. REFERENCES

References are provided in the IEEE format (11pt, normal print, with volume, pages, and year in bold), using the citations found in the source documents.

- [1] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “AutoCodeRover: Autonomous Program Improvement,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 24)*, New York, NY, USA, ACM, pp. 1592–1604, **2024**. doi: 10.1145/3650212.3680384.
- [2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” *arXiv:2405.15793 [cs.SE]*, **2024**.
- [3] X. L. K., Y. W., and L. Z., “Automated program repair in the era of large pre-trained language models,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, IEEE, pp. 1482–1494, **2023**.
- [4] S. A. J. S. T. D. R. T. L. S. W. E. C. B. C. B. P. N. T. K., and W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, “A survey on software fault localization,” *IEEE Transactions on Software Engineering*, vol. 42, no. 8, pp. 707–740, **2016**.

[5] **AutoCodeRover GitHub Repository**. Available:
<https://github.com/AutoCodeRoverSG/auto-code-rover>.

[6] J. H. N. E. T. S. K. D., and E. W. W., R. Gao, Y. Li, R. Abreu, and F. Wotawa, “SWE-bench: Towards a comprehensive benchmark for automated software engineering,” *arXiv:2310.19698 [cs.SE]*, **2023**.